

When does adapting a queue discipline’s parameters pay? A scaling law in target/RTT, measured on stock Linux

Amritha S^{*1}, Yugeshwaran P¹, and Deepti Annuncia¹

¹Department of Electronics and Communication Engineering, Vellore Institute of Technology, Chennai, 600127, Tamil Nadu, India

^{*}Corresponding author. E-mail: amritha16112005@gmail.com

Abstract

Linux ships `fq_codel` as its default queue discipline with four parameters fixed at configuration time. Whether adapting them while traffic runs is worth doing has been answered both ways, and neither answer states when the other holds. Across 210 runs spanning three workloads, eight queue disciplines and path round-trip times from 2 to 200 ms, the benefit is governed not by path RTT but by the dimensionless ratio $r = \text{target}/\text{RTT}$, following a logistic curve with half of its ceiling reached at $r = 0.50$ ($R^2 = 0.986$). The form holds across a 16-fold change in RTT and transfers to two further AQM mechanisms, but the ceiling does not: holding r at 1 while sweeping link rate 25-fold moves the benefit from 7.1 to 18.2 %. Published guidance places a correctly configured deployment at r between 0.05 and 0.10, five to ten times below the half-benefit point, making adaptation a remedy for misconfiguration rather than an improvement on correct configuration. Flow queueing with a delay target cuts tail latency 99.0 % below an unmanaged queue at indistinguishable goodput, and CAKE, which needs no controller, beats the adapted system at every RTT tested. All hypotheses and thresholds were registered before the data existed.

Keywords: active queue management; bufferbloat; `fq_codel`; CoDel; dimensionless scaling law; runtime parameter adaptation; Linux traffic control; pre-registered evaluation

1 Introduction

Bufferbloat occurs when buffers are larger than any control loop needs. The queueing delay rises and the throughput stays where it was[1]. Linux answers it with `fq_codel`, which divides traffic across roughly a thousand flow queues by Deficit Round Robin and gives each queue its own CoDel instance[2]. Four parameters govern it and none of them changes after configuration: `target` (5 ms by default), `interval` (100 ms), `limit` and `quantum`.

Adapting those parameters while traffic runs has obvious appeal and clear precedent. Adaptive RED adjusts a maximum drop probability by an additive-increase multiplicative-decrease rule[3]; Ye and Leung derive stability conditions for CoDel and adapt its `interval` analytically[4]; DESiRED adapts an AQM’s delay target in P4 using deep reinforcement learning and in-band telemetry[5]. Runtime adaptation of a delay target is therefore not novel, and no claim of novelty is made for it here.

Three families of queue discipline bear on this. Random Early Detection was one of the first methods in the area, dropping with a probability derived from average queue length[6]. Because it is sensitive to how it is configured, Adaptive RED followed. PIE takes a different route again, estimating delay and controlling it with a proportional-integral controller instead of working from queue length[7]. CoDel abandoned queue length altogether and drops when the minimum sojourn time over a sliding interval exceeds a target[8, 9]. Its authors argue that the two parameters are expressed relative to path round-trip time by construction and

therefore need no tuning. That claim is qualitative and has not, as far as we are aware, been given a boundary: it holds for a deployment whose target is scaled to its path and says nothing about one whose is not. Equation 1 supplies the boundary, which is the contribution here. The mechanism is old and the direction of the effect is unsurprising; what was missing is the number at which it begins to matter. Beyond `fq_codel`, CAKE integrates shaping, flow queueing and host fairness in one discipline[10], FQ-PIE is the corresponding combination for PIE[11], and L4S standardises a low-latency service class built on a dual-queue coupled AQM[12, 13]. All of these except L4S, whose dual-queue structure needs endpoint support this testbed does not provide, are measured here against one identical bottleneck, alongside an unmanaged queue and SFQ.

Among systems that adapt at runtime, QueuePilot learns a marking policy for small buffers by reinforcement learning[14] and AQM-LLM distils a language model into a marking controller[15]. Toopchinezhad and Ahmadi survey the machine-learning AQM literature and observe that heuristic schemes require careful parameter adjustment, which limits their applicability in deployment[16]. SCRR revisits fair-queueing scheduling for modern link rates[17], and Ray et al. characterise how the presence of an AQM distorts speed-test measurement[18]. Table 1 places the present work against this line. Each of these systems reports that its adaptation improved something, and each adapts a single quantity; none states the conditions under which that adaptation is worth running, which is the gap addressed here.

Table 1 The present work against six recent systems. Each adapts at most a single quantity and reports a gain for doing so; DualPI2 is included as the standardised alternative that adapts nothing. None states the condition under which adaptation is worth running.

System	Year	Quantity adapted	Method
Adaptive RED[3]	2001	maximum drop probability	AIMD
ACoDel[4]	2020	interval	analytical
QueuePilot[14]	2023	marking probability	offline RL
DualPI2 / L4S[13]	2023	none	dual queue
DESiRED[5]	2024	target delay	deep RL
AQM-LLM[15]	2025	action policy	distilled LLM
This work	2026	four parameters	AIMD with gradient

What that body of work does not supply is the condition under which adaptation pays. Systems report gains, independent evaluations report none, and neither camp states where the boundary between the two lies. RFC 7567 meanwhile recommends active queue management that requires no per-flow configuration[19], which makes a null result for adaptation a reportable outcome rather than a failed experiment. Two pieces of apparatus are missing throughout as well. A controller that merely occupies the processor is hard to distinguish from one that helps unless a third arm runs the same code and applies nothing; and the sparse probe traffic used to measure latency is usually averaged into the bulk flows, even where the discipline under test was designed to treat those two populations differently.

This study supplies the missing condition and the apparatus needed to establish it. The condition turns out to be a dimensionless ratio rather than any absolute quantity, which is what allows it to be stated once and applied to deployments of different speed and distance.

2 Results

Every value reported here is generated from the committed measurement logs by the analysis scripts; none is entered by hand. Unless stated otherwise, figures describe the steady workload at a 20 ms base path RTT over a 10 Mbit/s bottleneck, with means taken over three independent runs and uncertainties given as 95 % confidence intervals.

2.1 Comparison of the four measured outputs across all systems

Figure 1 sets out the four outputs the study measures, side by side, for all ten configurations. Tail latency of the sparse probe and mean latency of the bulk flows are reported separately throughout, because flow-queueing disciplines privilege sparse flows by construction and a single averaged latency conceals that. Queue occupancy is the mechanism those latencies arise from, and goodput is the quantity a latency improvement might have been bought with.

The dominant effect is the presence of flow queueing with a delay target, not the tuning of one. An unmanaged first-in-first-out queue reaches 2344 ms at the 95th percentile against 24.57 ms for static `fq_codel`, a 99.0 % reduction ($p < 0.001$), while goodput is statistically indistinguishable between them (9.453 against 9.414 Mbit/s, $p = 0.20$). The goodput panel is drawn from zero precisely so that this can be seen: every system delivers the same throughput, so the latency ordering across three orders of magnitude is a property of queue management and not a rate trade.

Fair queueing on its own is not sufficient. SFQ isolates flows but applies no delay target, and its bulk queues sit at their 127-packet limit throughout, so its sparse probe reads 30.68 ms while the bulk flows sharing the same link read 294.8 ms. The scheduler and the AQM do different jobs, and both are needed.

The three `fq_codel` arms, static, sham and adapted, are not separable from one another at this scale on any of the four outputs, which is the first indication that parameter tuning is a second-order effect.

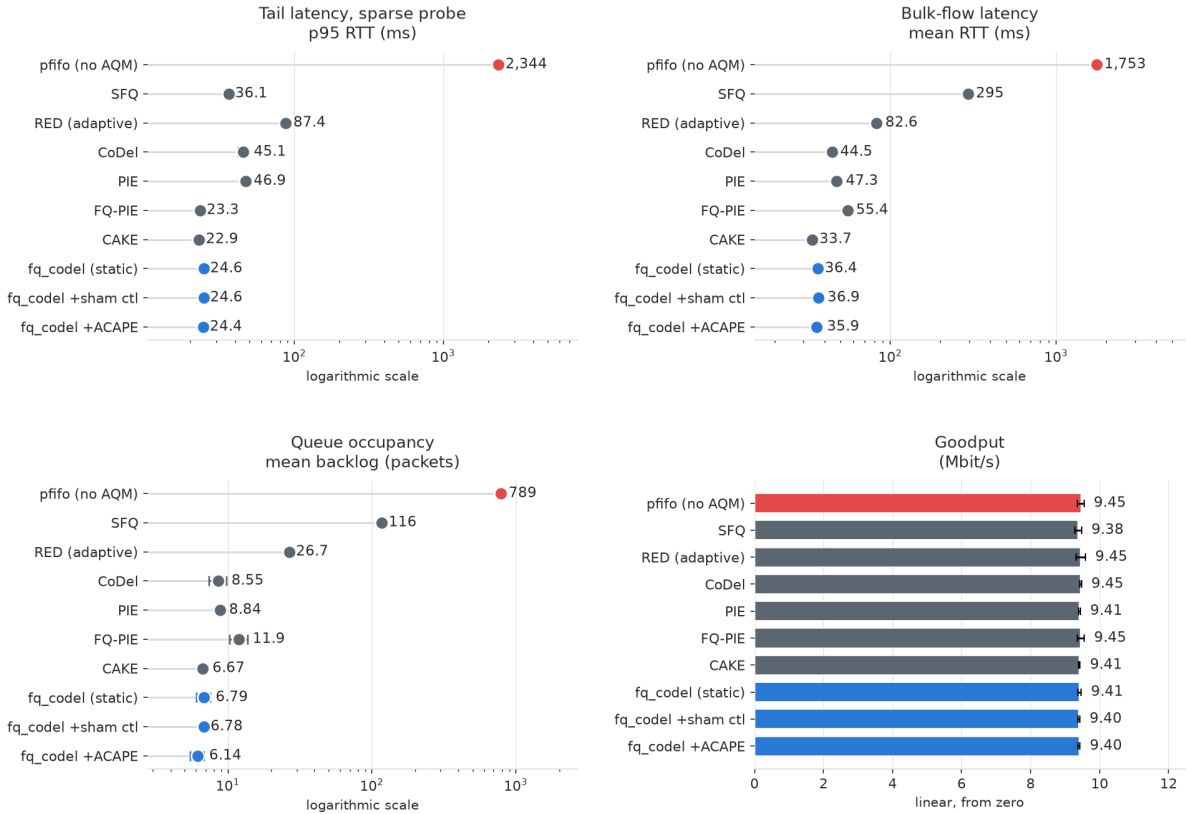


Fig. 1 The four measured outputs compared across all ten configurations under the steady workload. Lower is better for the three latency and occupancy panels, higher is better for goodput. Red marks the unmanaged baseline, blue the three `fq_codel` arms including the adapted system. The first three outputs span orders of magnitude and are drawn on a logarithmic axis, where position carries the value; goodput is linear from zero, where bar length carries it. Points and bars are means over three runs and whiskers are 95 % confidence intervals.

2.2 Effect of adaptation, parameter by parameter

Figure 2 gives every measured quantity against every system in one view, with each row’s rank shown as colour and each row’s true range printed beside it. The rank encoding makes within-row ordering legible across quantities whose units differ by orders of magnitude; the range column is there so that a row with strong colour contrast and no meaningful difference, such as goodput or fairness at $1.01\times$, is not read as a result.

Under the steady workload the largest movement the controller produces is in mean queue occupancy, from 6.79 to 6.14 packets. That is a 9.6 % reduction as a point estimate, but its 95 % confidence interval spans ± 1.00 packets and so covers zero, and the same is true of the other three outputs. Three repetitions per arm is not a design that can resolve small effects: at the seed-to-seed variation observed here it separates a benefit of about 3.5 % from zero, so these null results bound the effect rather than establish its absence. The sham arm narrows what the null can be attributed to: the static and sham arms also lie within each other’s intervals on all four outputs, so the controller’s computational cost is not detectable either, and the absence of an effect is not the cost of running the loop masking a benefit. Under the staged workload, built specifically to exercise adaptation by varying the flow count from two to twenty-four during the run, nothing separates there either, and the adjustments are seen to lag the transitions they respond to. Section 3 returns to why a null result is expected at this operating point.

The comparison that matters for practice is with CAKE, which requires no controller, no telemetry and no model of the queue. It reaches 22.9 ms at the 95th percentile against 24.43 ms for the adapted system. Replacing the discipline is worth more than tuning it.

2.3 A scaling law in `target`/RTT

The results above are taken at a single path RTT. Sweeping the base RTT from 2 to 200 ms while holding the configured `target` fixed shows that the benefit of adaptation is not a function of path RTT at all, but of the dimensionless ratio $r = \text{target}/\text{RTT}$. Fitting the reduction in bulk-flow RTT, b , over 10 cells gives

$$b(r) = \frac{B}{1 + (r_0/r)^k}, \quad B = 20.5\%, \quad r_0 = 0.50, \quad k = 2.6, \quad (1)$$

with $R^2 = 0.986$, where B is the ceiling, r_0 the ratio at which half of it is reached and k the sharpness of the transition. Three parameters over 10 cells will fit well whatever the shape, so R^2 is not the figure to judge this by. Refitting with one cell held out and asked to predict it, the form above gives a mean absolute error of 1.21 percentage points, against 2.41 for the same saturating form with the exponent fixed at one and 4.86 for a step that is flat below a threshold and flat above it. The third parameter earns its place out of sample, not merely in fit. Figure 3 shows the fit, and shows the same measurements plotted against path RTT alone, where they do not collapse.

Hypotheses and decision thresholds were committed to a public repository before any of these data were collected, and three of the four held. A crossover exists: benefit remains under 1.5 % and is not statistically significant below $r = 0.25$, and reaches 16.4 to 20.1 % at $r \geq 1$. The curve is invariant to path RTT at fixed ratio: cells at 5, 20 and 80 ms with $r = 1$ give 18.4, 17.6 and 16.4 %, a spread of 2.0 percentage points against a threshold of 5 set in advance. Leave-one-out prediction of the interior ratios has a mean absolute error of 1.4 percentage points, worst case 3.5. The functional form also transfers to mechanisms the fit never saw: refitted independently over the cells each mechanism has in common, plain `code1` places r_0 at 0.42 against `fq_code1`’s 0.49, and PIE, which shares nothing with CoDel beyond possessing a delay target, places it at 1.06, with R^2 of 0.978 and 0.992 respectively. The 0.49 here is the same quantity as the 0.50 of Equation 1, refitted on the seven cells common to all three mechanisms rather than the full 10; the small difference is the cost of the smaller fitting set and not a second estimate.

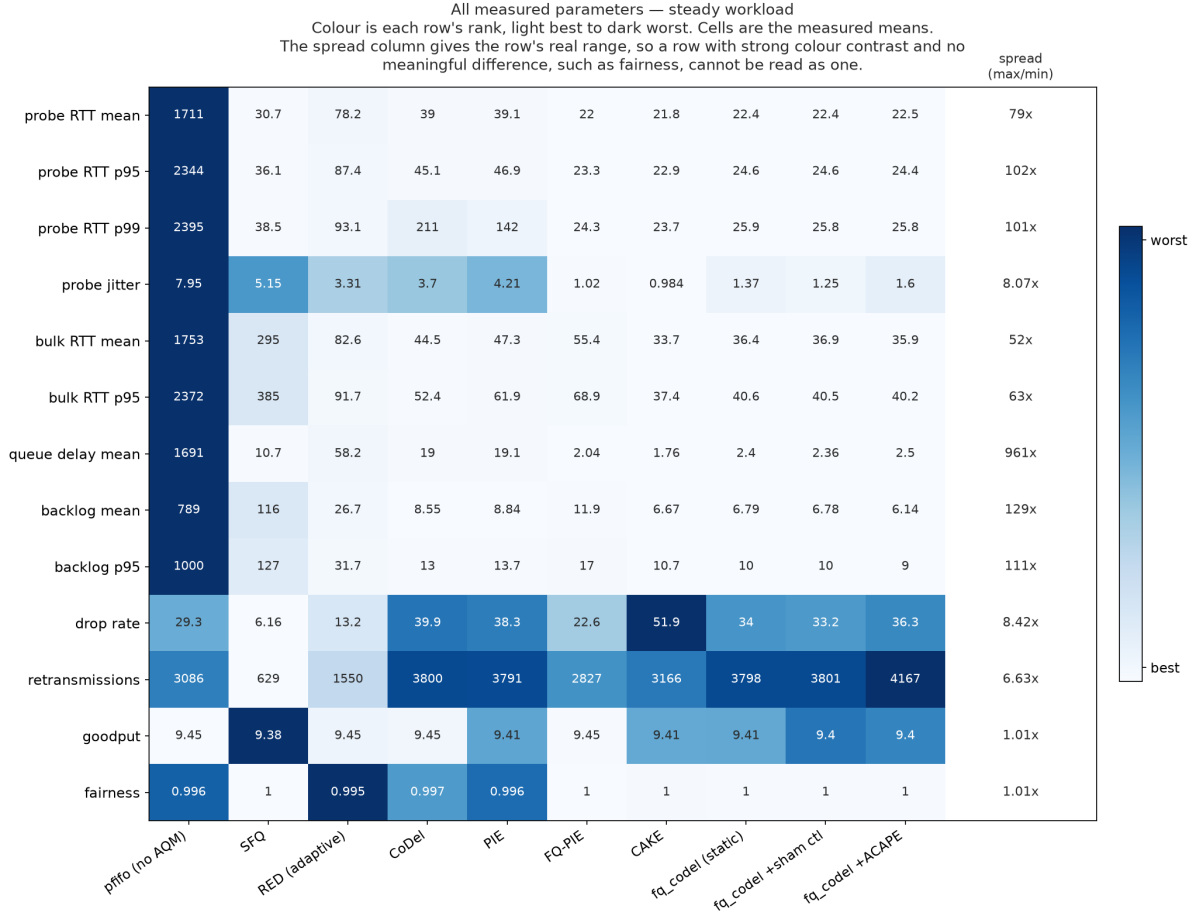


Fig. 2 Every measured quantity against every system under the steady workload. Colour encodes each row's rank, light best to dark worst, and cells carry the measured means. The spread column gives each row's true range as the ratio of its largest value to its smallest, so that rows with strong colour contrast but no meaningful difference are not mistaken for findings. Colour is comparable within a row and not between rows: two cells of the same shade in different rows can differ by a factor of eighty.

One pre-registered test failed and is reported as such. The spread in r_0 across the three mechanisms is $2.52\times$, beyond the $2\times$ threshold set in advance. It is driven entirely by PIE, the CoDel family agreeing to 0.49 against 0.42, and the three disciplines differ in their **target/interval** ratio as well as in mechanism, so the measurement is confounded. The functional form transfers; the constant does not.

2.4 The ratio fixes the shape, not the ceiling

The sweep above varies path RTT at a fixed 10 Mbit/s bottleneck, so it establishes that r governs the benefit along the RTT dimension and says nothing about the other one. A separate campaign supplies it: static and adapted pairs at 2, 5, 10, 20 and 50 Mbit/s, three repetitions each, all on a 5 ms path with the 5 ms default **target**. The ratio is therefore pinned at $r = 1$ in every cell, and any variation in benefit is variation in the ceiling itself.

Table 2 gives the result, and it is not invariant. Benefit ranges from 7.1 to 18.2 %, a spread of 11.0 percentage points, against the 5-point threshold this study used to call the RTT comparison consistent. Every cell is individually significant, so this is a real dependence and not sampling noise. It is also not monotonic: benefit peaks near $s \approx 2.1$ and falls away at both ends, where s is the configured target measured in units of the time one maximum-size packet takes to leave the interface.

This is the pre-registered second dimensionless group, and reporting it is what the earlier failure of that hypothesis amounts to in practice. The consequence for Equation 1 is specific: r determines where the transition sits and how sharp it is, and the fitted B of 20.5 % is the ceiling at 10 Mbit/s rather than a constant of the mechanism. An operator reading the predicate should take r as telling them whether adaptation can help and the rate column as telling them roughly how much.

Table 2 Benefit of adaptation at fixed $r = 1$ across a 25-fold change in link rate. All cells use a 5 ms path and the 5 ms default target, so only the rate changes. s is the target in units of one MTU’s serialisation time. Values are mean bulk-flow RTT $\pm$ 95 % confidence interval over three runs, and p is from Welch’s test.

Link rate (Mbit/s)	s	Static (ms)	Adapted (ms)	Benefit (%)	p
2	0.8	42.1 ± 1.6	36.6 ± 1.6	+13.2	0.0005
5	2.1	32.4 ± 0.5	26.5 ± 0.8	+18.2	<0.0001
10	4.1	28.1 ± 0.3	23.0 ± 0.4	+18.0	<0.0001
20	8.3	20.5 ± 0.3	17.6 ± 0.6	+14.1	0.0004
50	20.6	13.2 ± 0.2	12.3 ± 0.6	+7.1	0.0172

3 Discussion

Equation 1 has a direct operational reading. Published guidance places a correctly configured deployment at r between 0.05 and 0.10, while half the available benefit arrives only at $r = 0.50$, five to ten times outside that envelope. This also accounts for the null result of Section 2. Those measurements were taken at the Linux default **target** of 5 ms over a 20 ms path, which is $r = 0.25$, and Equation 1 puts the expected benefit there at 2.9 % against a measured 1.4 %, which three repetitions per cell cannot resolve. The controller was not failing in those measurements; it was being asked to work at a ratio where the law says there is little to recover. On a correctly configured link, therefore, there is almost nothing for parameter adaptation to recover, and the honest description of what such a controller does is that it repairs misconfiguration. The ratio is computable from two numbers an operator already has, the configured target and a path RTT estimate, so the predicate can be evaluated before any controller is deployed.

Where the ratio is small, the measurements point to a different action. Switching from

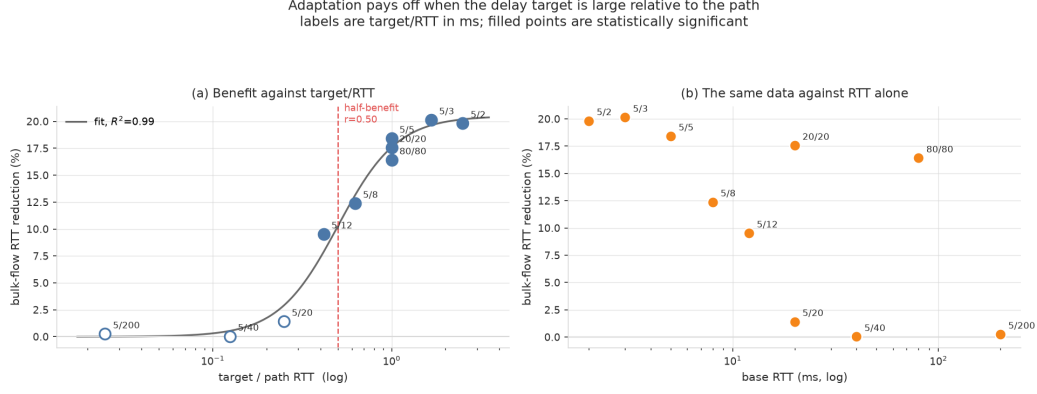


Fig. 3 Benefit of parameter adaptation against the dimensionless ratio $r = \text{target}/\text{RTT}$ (left) and against path RTT alone (right). The same measurements collapse onto a single logistic curve under the ratio and scatter under RTT, which is the evidence that the ratio rather than the delay is the governing variable. Filled points are statistically significant, open points are not.

`fq_codel` to CAKE is worth more at every RTT tested than tuning `fq_codel`'s four parameters, and it requires no runtime machinery at all. That the largest available improvement comes from a configuration change rather than from control is not a comfortable result for an adaptive system, but it is what the measurements show.

Several limits bound these claims. Network namespaces on one kernel are not a datacentre, and the ceiling B does move with link rate, as Section 2.4 measures directly: the dimensionless form is the portable claim and the fitted constants are not. Flow count was not varied and should be expected to matter in the same way. Three repetitions per cell give wide intervals, which is why the law is fitted across 10 cells rather than rested on any single one. The second dimensionless group of Section 2.4 was pre-registered with the prediction that benefit would be floored below unity; it instead peaks at intermediate values, so r alone does not close the problem and a complete predicate needs both groups. And the cross-mechanism transfer established the form but not the constant, as described above.

4 Methods

4.1 Testbed

Three Linux network namespaces model a client, a router and a server, connected by virtual Ethernet pairs. The router carries `netem` for base propagation delay and a token-bucket filter at 10 Mbit/s as its root queue discipline, with the discipline under test installed as the child of that shaper on the egress interface facing the server. Placing the bottleneck there puts it on the data path. An earlier arrangement used only two nodes and shaped the reverse direction. Every queue measurement taken that way described the acknowledgement path, not the data.

4.2 Queue disciplines evaluated

Nine configurations are compared under an identical bottleneck: `pfifo` with no AQM, SFQ, RED, `codel`, PIE, FQ-PIE, CAKE[10], static `fq_codel`, and `fq_codel` under the controller described below. CAKE is configured `besteffort` without its own bandwidth setting so that the token bucket remains the only shaper and every discipline meets the same bottleneck. A tenth arm is the sham controller. It runs at the same cadence and executes the same classification and prediction code, but changes no parameter, which is what separates the controller's processor cost from the effect of its decisions.

4.3 Controller

At each control tick the controller reads the queue discipline’s own statistics together with per-flow state maintained by an eBPF program attached at the `tc_clsact` egress hook[20], classifies a congestion regime from drop rate and backlog, and fits a trend to the last ten samples by least squares. At each adjustment opportunity it applies an additive-increase multiplicative-decrease rule taken from Adaptive RED[3], with $\beta = 0.9$ on decrease. Observation and adjustment run on separated timescales an order of magnitude apart, following the two-timescale stochastic approximation framework[21].

Every bound, step size and regime threshold is set relative to the configured `target`, not in absolute milliseconds. Suppose they were absolute. A controller started against an 80 ms target would then take steps sixteen times smaller, in relative terms, than one started against 5 ms, and the ratio dependence of Equation 1 would be partly an artefact of the controller’s own constants rather than a property of the queue.

Flow-map contents are read through the `bpf(2)` system call directly rather than by invoking an external utility per sample, which reduces the cost of one control tick from 3.17 to 0.709 s and avoids introducing a second process whose scheduling would confound the latency comparison the sham arm exists to isolate.

4.4 Measurements

Three latencies are recorded separately: a sparse ICMP probe for the latency a thin flow experiences, the smoothed round-trip time of the bulk TCP flows as reported by `iperf3`, and a queue-delay proxy computed from instantaneous backlog and the configured rate. They are not interchangeable, and the ratio between the sparse and the bulk latency ranges from 1.0 to 9.6 across the systems tested. Goodput is taken from `iperf3`’s received byte count and flow fairness is reported as Jain’s index[22].

4.5 Experimental design and statistics

Three workloads are used: a steady bulk load of eight TCP CUBIC[23] flows, a staged load whose flow count varies from two to twenty-four during the run so that the congestion regime changes, and a mixed load adding sparse constant-rate UDP probes alongside bulk traffic. Each cell is repeated three times as an independent run. Differences between conditions are assessed with Welch’s *t*-test, which does not assume equal group variances. Two-sided *p*-values come from Student’s *t* distribution through the regularised incomplete beta function rather than from a normal approximation. At three samples per group the approximation makes results look more significant than they are.

4.6 Platform

Experiments ran on Linux 6.12.48 under hardware-assisted virtualisation. The kernel is unmodified upstream source; the only departure from a distribution build is the configuration, which compiles every evaluated queue discipline in rather than as a module and enables BPF Type Format. No kernel code was patched, which is what *stock Linux* in the title refers to: the disciplines under test are the ones a deployment would get.

Data availability

All code, kernel configuration, raw measurement logs and analysis scripts are available at <https://github.com/Amritha902/ccn-linux-qdisc-study>. Every table and every numerical value quoted in this article is generated directly from the committed logs by the analysis scripts; none is entered by hand. The pre-registered hypotheses and their decision thresholds are committed to the same repository with timestamps preceding the corresponding measurement runs. The

repository holds 292 runs in total; the 210 cited here are the four campaigns this article draws on, and the remainder belong to two further pre-registered studies, of the controller’s runtime RTT estimator and of a self-gating variant, which are reported separately and do not bear on the claims made above.

Declarations

Funding. No funding was received for conducting this study.

Conflict of interest. The authors declare no competing interests.

Ethics approval and consent to participate. Not applicable.

Consent for publication. Not applicable.

Materials availability. Not applicable.

Code availability. All analysis and experiment code is available at the repository given above.

Author contribution. A.S. designed and implemented the testbed, the controller and the analysis pipeline, and wrote the manuscript. Y.P. contributed to the experimental design and the measurement campaigns. D.A. supervised the work. All authors reviewed the manuscript.

References

- [1] J. Gettys and K. Nichols. Bufferbloat: dark buffers in the Internet. *Commun. ACM*, 55(1):57–65, 2012.
- [2] T. Høiland-Jørgensen, P. McKeeney, D. Täht, J. Gettys, and E. Dumazet. The FlowQueue-CoDel packet scheduler and active queue management algorithm. Technical Report RFC 8290, IETF, 2018.
- [3] S. Floyd, R. Gummadi, and S. Shenker. Adaptive RED: an algorithm for increasing the robustness of RED’s active queue management. Technical report, ICSI, 2001.
- [4] J. Ye and K.-C. Leung. Adaptive and stable delay control for combating bufferbloat: theory and algorithms. *IEEE Syst. J.*, 14(1):1285–1296, 2020.
- [5] F. Rodriguez et al. DESiRED: dynamic, enhanced and smart iRED, a P4-AQM with deep reinforcement learning and in-band network telemetry. *Comput. Netw.*, 244, 2024.
- [6] S. Floyd and V. Jacobson. Random early detection gateways for congestion avoidance. *IEEE/ACM Trans. Netw.*, 1(4):397–413, 1993.
- [7] R. Pan, P. Natarajan, F. Baker, and G. White. Proportional integral controller enhanced (PIE). Technical Report RFC 8033, IETF, 2017.
- [8] K. Nichols and V. Jacobson. Controlling queue delay. *ACM Queue*, 10(5):20–34, 2012.
- [9] K. Nichols, V. Jacobson, A. McGregor, and J. Iyengar. Controlled delay active queue management. Technical Report RFC 8289, IETF, 2018.
- [10] T. Høiland-Jørgensen, D. Täht, and J. Morton. Piece of CAKE: a comprehensive queue management solution for home gateways. In *Proc. IEEE LANMAN*, 2018.
- [11] G. Ramakrishnan, M. Bhasi, V. Saicharan, L. Monis, S. D. Patil, and M. P. Tahiliani. FQ-PIE queue discipline in the Linux kernel: design, implementation and challenges. In *Proc. IEEE LCN*, 2019.
- [12] B. Briscoe, K. De Schepper, M. Bagnulo, and G. White. Low latency, low loss and scalable throughput (L4S) Internet service: architecture. Technical Report RFC 9330, IETF, 2023.

- [13] K. De Schepper, B. Briscoe, and G. White. Dual-queue coupled active queue management for L4S. Technical Report RFC 9332, IETF, 2023.
- [14] M. Dery, O. Krupnik, and I. Keslassy. QueuePilot: reviving small buffers with a learned AQM policy. In *Proc. IEEE INFOCOM*, 2023.
- [15] S. R. Pokhrel, D. Satish, J. Kua, and A. Walid. Distilling large language models for network active queue management. *IEEE Trans. Netw.*, 2025.
- [16] M. P. Toopchinezhad and M. Ahmadi. Machine learning approaches for active queue management: a survey, taxonomy and future directions. *Comput. Netw.*, 262, 2025.
- [17] E. Sharafzadeh, R. Matson, J. Tourrilhes, P. Sharma, and S. Ghorbani. Self-clocked round-robin packet scheduling. In *Proc. USENIX NSDI*, 2025.
- [18] S. Ray, T. Sharma, J. Marques, P. Schmitt, F. Bronzino, and N. Feamster. Characterizing the impact of active queue management on speed test measurements. *arXiv:2511.19213*, 2025.
- [19] F. Baker and G. Fairhurst. IETF recommendations regarding active queue management. RFC 7567, IETF, July 2015. BCP 197.
- [20] C. Hung and P. Wang. eBPF qdisc: a generic building block for traffic control. In *Netdev 0x17*, 2023.
- [21] V. S. Borkar. Stochastic approximation with two time scales. *Syst. Control Lett.*, 29(5):291–294, 1997.
- [22] R. Jain, D.-M. Chiu, and W. Hawe. A quantitative measure of fairness and discrimination for resource allocation in shared computer systems. Technical Report TR-301, DEC, 1984.
- [23] S. Ha, I. Rhee, and L. Xu. CUBIC: a new TCP-friendly high-speed TCP variant. *ACM SIGOPS Oper. Syst. Rev.*, 42(5):64–74, 2008.